



UNIVERSITÉ
LAVAL

UNIVERSITÉ LAVAL

GIF-7001

Vision Numérique

Rapport Final

Détection et suivi du visage pour contrôle par oculométrie

présenté à

Robert Bergevin et Félix-Antoine Demers

par l'équipe 16

Membres de l'équipe	Matricules	Remarques
Axel François	537407412	S.O
Lauris Boatti-Barusco	537393107	N'a pas participé au travail de l'équipe
Saad ElGhissassi	537393180	S.O
Théo Fornier	537396160	S.O
Wilkins Galan	537396167	S.O

Table des matières

1	Mise en contexte et description de la solution réalisée	3
2	Justification des choix de design finaux	5
3	Résultats finaux obtenus	6
4	Améliorations futures suggérées	8
5	Vidéo narrée de démonstration finale	9

1 Mise en contexte et description de la solution réalisée

Les interfaces homme-machine évoluent vers des modalités non intrusives exploitant les micro-mouvements oculaires, la pose de la tête et les expressions faciales comme signaux de commande. Ces approches sont centrales pour l’accessibilité (contrôle mains libres), la sécurité automobile (sommolence, distraction), la réalité étendue (fovéation, immersion) et les neurosciences (attention visuelle, saccades).

Dans ce contexte, nous avons développé une chaîne de traitement multimodale temps réel qui réalise, à partir d’une simple webcam, (i) le contrôle directionnel d’un curseur par le regard et (ii) l’animation faciale d’un avatar stylisé piloté par les pupilles, la pose de tête, l’ouverture des paupières et de la bouche ainsi que plusieurs expressions faciales. L’ensemble du système est implémenté dans un programme Python unique avec plusieurs modes d’exécution. La chaîne de traitement repose principalement sur les bibliothèques MediaPipe FaceMesh et OpenCV.

L’ensemble de la chaîne de traitement peut se résumer par la séquence suivante :

$$\text{Webcam} \xrightarrow{\text{FaceMesh}} \mathcal{L} \xrightarrow{\text{extraction yeux+pose}} \mathbf{f}_t \xrightarrow{\text{module regard}} (x_n, y_n, s_t) \xrightarrow{\text{sortie}} \text{curseur ou avatar.}$$

Tout d’abord, la cohérence géométrique de la pose de tête et des coordonnées oculaires repose sur une calibration intrinsèque standard à partir d’un échiquier filmé sous différents angles.

On considère des points 3D planaires connus $P_i = (X_i, Y_i, 0)^\top$ et leurs projection 2D associées $p_i = (u_i, v_i)^\top$. Le modèle sténopé s’écrit $s_i(u_i, v_i, 1)^\top = K[R \mid t](X_i, Y_i, 0, 1)^\top$, où la matrice intrinsèque K décrit les paramètres optiques, tandis que R et t représentent la pose de l’échiquier. OpenCV estime donc K , les coefficients de distorsion d et la pose par vue en minimisant la somme des erreurs de reprojection $\min_{K,d,R_k,t_k} \sum_k \sum_i \|p_{i,k} - \hat{p}_{i,k}(K, d, R_k, t_k)\|_2^2$. Les paramètres calibrés sont alors sauvegardés dans **intrinsics.json**, et un mode dédié (**undistort**) rectifie ensuite le flux vidéo afin de stabiliser la géométrie de la région oculaire et fiabiliser les estimations de l’iris et de la pose de tête.

Chaque image du flux webcam est ensuite traitée par MediaPipe, qui renvoie un ensemble de repères faciaux $(x_k, y_k) \in [0, 1]^2$: $\mathcal{L} = \{\ell_k = (x_k, y_k, z_k) \mid k = 0, \dots, 467\}$ en coordonnées normalisées.

À partir de sous-ensembles d’indices (paupières, canthus, iris), le programme construit pour chaque œil une ROI (région d’intérêt) rectangulaire en pixels : concrètement, pour l’œil gauche, à partir d’un ensemble I_L , les points sont projetés en pixels via la résolution (W, H) , puis on calcule le plus petit rectangle englobant $R_L = [x_{\min}^L, x_{\max}^L] \times [y_{\min}^L, y_{\max}^L]$, légèrement élargi par des facteurs d’échelle **scale_x**, **scale_y**. On procède de manière identique pour l’œil droit R_R .

L’iris est estimée par le barycentre des repères dédiés, puis ses coordonnées sont normalisées dans chaque ROI selon $c_x^{\text{œil}} \leftarrow \frac{c_x^{\text{œil}} - x_{\min}^L}{x_{\max}^{\text{œil}} - x_{\min}^L}$, $c_y^{\text{œil}} \leftarrow 1 - \frac{c_y^{\text{œil}} - y_{\min}^{\text{œil}}}{y_{\max}^{\text{œil}} - y_{\min}^{\text{œil}}}$, avec $\text{œil} = \{L, R\}$, de sorte que $c_y = 1$ corresponde au regard vers le haut et $c_y = 0$ vers le bas.

En parallèle, le taux d’ouverture de l’œil (EAR) est calculé pour chaque œil à partir de repères de paupières. Ce score permet d’écarter les trames où l’utilisateur cligne ($\text{EAR} < \text{seuil}$) et fournit un indice complémentaire du mouvement vertical, les yeux étant plus ouverts lorsque le regard est dirigé vers le haut. Finalement, les quadruplets $(c_x^L, c_y^L, c_x^R, c_y^R)$, enrichis d’angles de pose et d’EAR, sont mis à jour à 30–60 fps et constituent le cœur du descripteur oculaire. En complément de l’extraction oculaire, la pose 3D de la tête est estimée via un problème

PnP posé sur un sous-ensemble de repères faciaux stables (bout du nez, menton, coins externes des yeux, commissures de la bouche). Pour cela, on définit un modèle 3D simplifié $\mathcal{P} = \{P_j = (X_j, Y_j, Z_j)^\top\}_{j=1..6}$, $\mathcal{Q} = \{q_j = (u_j, v_j)^\top\}_{j=1..6}$, où les q_j sont extraits de $\mathcal{L}$. OpenCV résout alors $\min_{R,t} \sum_j \|q_j - \pi(K, d, RP_j + t)\|_2^2$, où π applique la projection perspective et la distorsion (en mode **headpose**, en réutilisant K, d de **intrinsics.json**), puis le vecteur de rotation **rvec** est converti en angles d'Euler (yaw, pitch, roll). Nous exploitons surtout yaw (rotation horizontale) et pitch (rotation verticale), utilisés comme variables auxiliaires pour désambiguïser certains motifs oculaires, comme composantes de $\mathbf{f}_t$ et comme paramètres de micro-rotation pour l'avatar.

À ce stade, pour chaque trame validée, le descripteur $\mathbf{f}_t = (c_x^L, c_y^L, c_x^R, c_y^R, \text{yaw}, \text{pitch}, \text{EAR})^\top \in \mathbb{R}^7$ combinant yeux, pose et état des paupières est construit. Ce vecteur alimente un module directionnel discret (**gaze_directional**), qui détecte neuf directions de regard (centre, quatre cardinales, quatre diagonales) à partir de déplacements relatifs par rapport à une référence.

Nous n'apprenons pas une signature par état, mais uniquement une position neutre correspondant au regard centré, puis nous travaillons en déplacements relatifs. Ainsi, dans la phase de calibration (**gaze_directional**), l'utilisateur fixe le centre de l'écran pendant quelques secondes, chaque trame valide fournit un $\mathbf{f}_t$ dont on conserve $(c_x^L, c_y^L, \text{EAR})$, puis on calcule les moyennes $c_{x,\text{centre}}^L, c_{y,\text{centre}}^L, \text{EAR}_{\text{centre}}$, qui définissent la position moyenne de la pupille gauche et l'ouverture moyenne des yeux au regard neutre.

Ensuite, en exploitation, pour chaque nouvelle trame, nous dérivons les déplacements relatifs $\Delta x_t = c_x^L(t) - c_{x,\text{centre}}^L$, et $\Delta y_t = (c_y^L(t) - c_{y,\text{centre}}^L) \gamma$, où $\gamma > 1$ (dans le code **AMPLIF_Y=2.0**) amplifie les mouvements verticaux, plus limités que les mouvements horizontaux. La variation d'ouverture des yeux est quant à elle exploitée via $\Delta \text{EAR}_t = (\text{EAR}_t - \text{EAR}_{\text{centre}}) \kappa$, avec un facteur κ d'amplification. Ainsi, la composante verticale utilisée pour la classification est alors $\text{score}_y(t) = \Delta y_t + \Delta \text{EAR}_t$, ce qui permet de mieux distinguer un vrai regard haut (paupières plus ouvertes) d'un simple bruit vertical.

À partir de Δx_t et de $\text{score}_y(t)$, le système affecte chaque trame à l'un des neuf états directionnels $\mathcal{S} = (\text{CENTRE}, \text{HAUT}, \text{BAS}, \text{GAUCHE}, \text{DROITE}, \text{HAUT-GAUCHE}, \text{HAUT-DROITE}, \text{BAS-GAUCHE}, \text{BAS-DROITE})$. Plus précisément, la composante horizontale est seuilée avec un paramètre τ_x : si $\Delta x_t > \tau_x$ la tendance est GAUCHE, si $\Delta x_t < -\tau_x$ elle est DROITE, sinon l'état est CENTRE horizontalement. De même, la composante verticale est évaluée via $\text{score}_y(t)$ et un seuil τ_y : si $\text{score}_y(t) > \tau_y$ la tendance est HAUT, si $\text{score}_y(t) < -\tau_y$ elle est BAS, sinon l'état reste CENTRE verticalement. Par conséquent, la combinaison fournit directement les neuf états, les diagonales émergeant comme la combinaison **d'un déplacement** horizontal et vertical significatifs, à la manière d'un joystick analogique.

Afin d'améliorer la stabilité, deux mécanismes temporels sont mis en œuvre : d'abord, un lissage One-Euro 2D est appliqué sur la sortie (x_n, y_n) , chaque état $s \in \mathcal{S}$ étant associé à une position écran normalisée, par exemple $\text{CENTRE} \mapsto (0,5,0,5)$ ou $\text{GAUCHE} \mapsto (0,1,0,5)$. Ensuite, un mécanisme de gel du curseur est déclenché par un clin d'œil volontaire : l'EAR est calculé séparément pour l'œil gauche et l'œil droit, et si un seul des deux passe sous le seuil (un œil fermé, l'autre ouvert) ce motif est interprété comme un geste volontaire. Le curseur est alors affiché en gris et figé jusqu'à ce que les deux yeux soient de nouveau ouverts ou fermés, auquel cas il redevient actif (rouge).

Au final, cette logique fournit un comportement directionnel interprétable, réactif et robuste pour le contrôle par oculométrie, reposant sur des méthodes de vision par ordinateur classiques

plutôt que sur de l'apprentissage automatique.

Enfin, l'application intègre un avatar facial rendu en temps réel à partir du maillage 3D fourni par MediaPipe FaceMesh.

Pour cela, le système exploite les 468 repères faciaux et le maillage triangulaire : pour chaque arête (k, l) , un segment est tracé entre les projections des points sur un canevas 2D, et la profondeur relative (via z_k, z_l) module légèrement la teinte (zones proches plus sombres, zones éloignées plus claires). Les yeux sont modélisés par des polygones remplis pour le blanc de l'œil, une iris circulaire colorée et une pupille noire dont la position dépend de (x_n, y_n) , tandis que l'ouverture des paupières est pilotée par l'EAR. La bouche est dessinée à partir des contours des lèvres, avec un ratio d'ouverture qui pilote l'animation et une ligne médiane pour séparer les lèvres, tandis que les sourcils sont animés en fonction de leur distance aux yeux.

En greffant un module heuristique aux mesures différentes mesures d'animation du visage (EAR par œil, coins de bouche plus hauts ou plus bas que le centre, ouverture de bouche et indices liés au nez ou aux sourcils), nous identifions quelques expressions (SOURIRE, TRISTESSE, COLÈRE, CLIN_OEIL_GAUCHE, CLIN_OEIL_DROIT, SURPRISE, BOUCHE_OUVERTE, YEUX_FERMÉS, NEUTRE).

L'expression courante et un score de confiance sont alors affichés dans la fenêtre avatar et dans la fenêtre caméra, et l'avatar adapte légèrement certaines couleurs (lèvres, surimpressions) en fonction de l'expression, ce qui crée une boucle visuelle immédiate entre les mouvements faciaux de l'utilisateur et le comportement de l'avatar.

2 Justification des choix de design finaux

Le design final résulte d'un compromis réfléchi entre réalisme géométrique, stabilité des signaux, charge de calcul et interprétabilité.

Choix des outils et bibliothèques Le langage Python a été retenu pour sa lisibilité et l'écosystème scientifique mature. OpenCV constitue l'ossature **du pipeline** (capture, calibration, solvePnP, rendu), tandis que MediaPipe FaceMesh fournit 468 repères fiables, incluant l'iris, à 30–60 fps sans entraînement préalable. Cette combinaison assure portabilité, reproductibilité et exécution temps réel sur une simple webcam.

Calibration de la caméra La calibration intrinsèque via échiquier imprimé sur papier est une manière "grand public" **d'obtenir** la matrice interne K , les coefficients de distorsion et une mesure objective de qualité (erreur RMS). Ces paramètres stabilisent ensuite l'estimation de la pose via **solvePnP**. De plus, la rectification du flux améliore la cohérence géométrique des régions oculaires et réduit ainsi la variabilité inter-sessions.

Détection et suivi oculaires Le choix de s'appuyer sur les repères MediaPipe plutôt que sur des traitements d'intensité a permis une invariance lumineuse, un coût de calcul constant et une normalisation fiable dans les régions d'intérêt. En particulier, la normalisation des centres pupillaires $(c_x^L, c_y^L, c_x^R, c_y^R)$ réduit les effets de la distance caméra et rend les mesures plus stables. Étant donné que les signaux pupillaires, d'ouverture de paupières et de pose sont bruités, le filtre One-Euro a été retenu car il atténue les oscillations rapides tout en préservant la réactivité, ce qui est essentiel pour le contrôle directionnel.

Estimation de la pose de tête L’estimation de la pose 3D par `solvePnP`, fondée sur des repères anatomiquement stables, permet de garantir une solution géométriquement contrainte exploitant pleinement la calibration (K, d) . Les angles yaw/pitch obtenus sont ainsi cohérents entre utilisateurs et sessions. Ils améliorent à la fois l’interprétation des signaux oculaires (distinction entre mouvement de tête et mouvement du regard) et l’animation de l’avatar, en restituant des micro-rotations naturelles.

Modélisation directionnelle du regard Deux approches ont été envisagées : une régression quadratique continue et un modèle directionnel discret. La première, sensible aux dérives et exigeante en calibration, a été abandonnée au profit d’un automate à neuf états fondé sur les déplacements relatifs $(\Delta x_t, \Delta y_t)$ de la pupille, une amplification verticale et des seuils (τ_x, τ_y) . Ce modèle est robuste aux translations globales du visage, explicable et déterministe, tout en restant compatible avec les latences du système. La régression continue demeure disponible en module séparé pour des usages nécessitant une interpolation fine.

Architecture modulaire et avatar Le programme est structuré en modes indépendants (calibration, extraction faciale, suivi pupillaire, estimation de pose, clignement, regard continu, regard directionnel, avatar), ce qui simplifie le débogage et l’évaluation. L’avatar exploite directement les repères MediaPipe, le maillage triangulaire complet, les pupilles normalisées, le taux d’ouverture des yeux, yaw/pitch, l’ouverture labiale et la distance sourcils–yeux. L’utilisation du maillage complet assure une cohérence visuelle naturelle, sans coût supplémentaire, puisque les repères sont déjà disponibles. Finalement, le filtre One-Euro appliqué aux paramètres d’animation garantit un rendu fluide tout en respectant les contraintes temps réel (30–60 fps).

3 Résultats finaux obtenus

Notre système a été testé sur un MacBook Pro M1 avec webcam 720p dans des conditions contrôlées (éclairage constant, distance d’environ 55 cm, 28–30 FPS). L’objectif était d’évaluer la cohérence de la chaîne capture → détection → estimation géométrique → interaction. Les expérimentations montrent que la stabilité des modules bas niveau (calibration, pose, pupille) se traduit par un comportement global cohérent du modèle directionnel et de l’avatar.

Calibration géométrique et estimation de la pose L’évaluation de la calibration intrinsèque (Table 1), réalisée à partir de 11 vues d’un échiquier 7×7 (carrés de 18 mm) montre une erreur moyenne (≈ 0.21 px) et maximale (≈ 0.35 px) permettant de conclure que K et d sont suffisamment précis pour l’estimation de pose, le module de regard et l’avatar.

L’évaluation de la stabilité de la pose en mode `headpose` sur 163 images avec la tête immobile (Table 2) montre quant à elle des variations angulaires ($< 2^\circ$) et une latence très faibles (0.33 ms), preuve que la pose est exploitable en temps réel pour le modèle directionnel et pour l’avatar.

Mesure	Valeur
Erreur RMS (OpenCV)	1.509 px
Erreur moyenne de reprojection	0.209 px
Erreur maximale par vue	0.352 px
Amplitude estimée de la distorsion	$\approx 23\%$

Table 1: Performance de la calibration intrinsèque.

Mesure	Valeur
$\sigma(\text{yaw}), \sigma(\text{pitch})$	$0.33^\circ, 1.88^\circ$
Erreur moyenne de reprojection	3.03 px
Erreur maximale	3.31 px
Latence moyenne par estimation	0.33 ms

Table 2: Stabilité et performance de l’estimation de pose.

Détection oculaire : stabilité, bruit et précision Les performances de la brique oculaire (Table 3) montrent une fréquence de détection d’environ 28 FPS. Le post-traitement coûte en moyenne 0.67 ms par trame (max 2.20 ms), très en dessous du budget de 33 ms à 30 FPS : le coût dominant est donc FaceMesh, et non les opérations spécifiques au regard. La variabilité intra-ROI (0.0145, soit $\approx 1.5\%$ de la taille de l’œil) reflète à la fois bruit de détection et micro-saccades. Le filtre One-Euro réduit la gigue (le ”jitter”) de 0.0116 à 0.0014 (facteur 8.38), ce qui stabilise le curseur tout en conservant une bonne réactivité.

Mesure	Valeur
Fréquence de détection	28.0 FPS
Post-traitement (ROI + caractéristiques)	0.67 ms/trame
Variabilité intra-ROI du centre de pupille	0.0145 (normalisée)
Gigue pupillaire avant filtrage	0.0116
Gigue pupillaire après One-Euro	0.0014
Réduction du bruit	facteur 8.38

Table 3: Performances de détection oculaire et stabilité.

Précision de localisation. L’erreur normalisée de localisation de la pupille est définie par

$$e_t = \sqrt{(c_x - c_x^{\text{ref}})^2 + (c_y - c_y^{\text{ref}})^2}, \quad \bar{e} = \frac{1}{T} \sum_{t=1}^T e_t.$$

Nous l’avons évaluée sur une vidéo annotée manuellement ($T = 342$ trames), ce qui nous a conduit à $\bar{e} = 0.0157$ et $\sigma_e = 0.0122$, soit une erreur moyenne d’environ 1.6 % de la taille de la ROI, suffisante pour un contrôle de curseur de précision modérée (défilement grossier du curseur à travers l’écran), mais insuffisante pour de la sélection très fine (pointage ciblé d’un point particulier avec les yeux) sans agrandissement ou recalibration locale.

Modèle directionnel : performance et robustesse Le modèle directionnel à neuf états a été évalué sur des séquences de transitions imposées, chaque changement étant déclenché par un clignement volontaire (Table 4) : nous remarquons que directions haut, bas et diagonales atteignent des taux de reconnaissance $> 90\%$, tandis que l’état neutre et les transitions purement horizontales restent plus sensibles au bruit (déplacement pupillaire du même ordre que la gigue résiduelle). La stabilisation après clignement est rapide (environ 100–150 ms) et le taux de faux positifs reste inférieur à 4 %, ce qui est suffisant pour de la navigation ou de la sélection grossière.

Transition	Reconnaissance	Erreur moyenne
Centre → Droite	58 %	0.42
Centre → Gauche	45 %	0.55
Centre → Haut	92 %	0.09
Centre → Bas	89 %	0.12
Diagonales	90–94 %	0.08–0.11

Table 4: Performance directionnelle lors de transitions guidées par clignements.

Avatar : qualité et performances L’avatar exploite le maillage FaceMesh, les pupilles normalisées, l’EAR et une mesure d’ouverture labiale. L’analyse de ses performances (Table 5) révèle une latence totale de l’ordre de 50 ms, compatible avec une interaction fluide. Les clignements sont détectés avec un taux d’environ 95 %, et l’erreur sur l’ouverture labiale (≈ 0.08)

permet de distinguer de manière stable bouche fermée, entrouverte ou ouverte. Environ 88 % des trames présentent un regard cohérent lors de la fixation d'un point; les autres sont affectées par micro-saccades, bruit ou petits mouvements résiduels. Les expressions fines (comme COLÈRE) restent plus difficiles à restituer, la résolution des repères de sourcils étant limitée.

Mesure	Valeur
Temps de rendu avatar	52.7 ms/trame
Fréquence d'affichage	122 FPS
Latence chaîne de traitement complète	≈ 53 ms
Taux de détection des clignements	95 %
Erreur sur l'ouverture labiale	0.08 (normalisée)
Trames avec regard cohérent	88 %

Table 5: Performances et qualité de rendu de l'avatar.

Synthèse des résultats et limites Il vient que globalement, le système atteint une calibration géométrique précise (erreur de reprojection ≈ 0.2 px), une localisation pupillaire fiable ($\bar{e} \approx 0.016$ normalisé), un contrôle directionnel avec des taux de reconnaissance moyens autour de 85 % (et > 90 % pour les directions verticales et diagonales) et un avatar fluide avec une latence ≈ 50 – 60 ms sur CPU standard. Les principales limites tiennent à l'usage d'une webcam RVB (reflets, amplitude verticale réduite, dépendance à l'éclairage, expressions fines peu marquées), mais la faisabilité d'une interface de pointage et d'animation faciale reposant uniquement sur des briques géométriques et une webcam est largement démontrée.

4 Améliorations futures suggérées

Plusieurs axes d'amélioration émergent naturellement de l'analyse des performances obtenues. Elles concernent notamment un raffinement de la géométrie, de la précision pupillaire, en cohérence avec l'évolution récente des systèmes d'interaction par oculométrie sur webcam.

Modélisation et calibration du regard Le modèle directionnel, fondé sur des déplacements 2D dans la ROI, pourrait être étendu par une modélisation 3D de l'œil (pose de tête, calibration, modèle sphérique) afin d'estimer un véritable rayon de regard et son intersection avec l'écran. Une calibration plus dense, associée à un mappage polynomial ou régularisé, améliorerait la prédiction continue (x_n, y_n) . Un modèle hybride — états discrets pour la robustesse, interpolation pour le raffinement — rejoindrait les approches des dispositifs commerciaux (Tobii4C, Dynavox).

Détection d'expressions Il serait utile d'ajouter des modules capturant les micro-déformations des sourcils et des paupières (haussement, froncement glabellaire, abaissement palpébral).

Architecture temps réel unifiée Fusionner la chaîne capture–FaceMesh–regard–avatar dans une boucle unique et la rendre persistante réduirait la latence et stabiliserait le débit, conformément aux architectures modernes de visioconférence augmentée (comme Zoom Avatars).

Évaluation expérimentale plus systématique Enfin, une validation plus complète du système pourrait inclure : mesures d'erreurs angulaires sur des points cibles, étude de la dérive et de la variabilité inter-utilisateurs, et tâches de pointage de type loi de Fitts, conformément aux références actuelles pour l'évaluation objective des interfaces basées sur le regard.

5 Vidéo narrée de démonstration finale

La vidéo se trouve à l'adresse suivante : <https://youtu.be/0Fc2Pcbqubg>